TÀI LIỆU HỌC TẬP — DÀNH CHO SINH VIÊN

BUỔI 1: NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Làm quen Google Colab & Ôn tập Python nền tảng

Họ và tên: Nguyễn Quách Phú Tài	MSSV: 123000609	Nhóm: ……	Ngày: 8/9/2026

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

- Tài liệu này đã có sẵn toàn bộ nội dung/code cho từng hoạt động — bạn không cần tự viết code từ đầu. Quy trình cho mỗi hoạt động là: 📋 Copy code có sẵn → ▶️ Paste vào Colab và chạy → 💭 Suy nghĩ/quan sát → 🔧 Cải tiến (tự thử thay đổi) → 🌾 Thu hoạch (ghi lại điều học được)
- Ở bước 🔧 Cải tiến, hãy dán phần code bạn đã tự chỉnh sửa vào khung viền vàng đứt nét; ở bước 🌾 Thu hoạch, hãy chụp màn hình (screenshot) kết quả và dán vào khung viền xám đứt nét (🖼️)
- Ghi câu trả lời/suy nghĩ của bạn vào các dòng kẻ chấm — không cần viết dài, chỉ cần thật (đây không phải bài kiểm tra đúng/sai)
- Cuối Tiết 4, hoàn thành phần "BÁO CÁO TRẢI NGHIỆM" ở cuối tài liệu này
- Trước khi rời lớp: vào File → Download → PDF Document để xuất toàn bộ tài liệu (đã có code cải tiến + ảnh + ghi chú của bạn) thành file PDF và nộp theo hướng dẫn của giảng viên

MỤC TIÊU BUỔI HỌC

Sau buổi học này, bạn sẽ có khả năng:

- Phân biệt được AI, Machine Learning (ML), Deep Learning (DL) qua ví dụ cụ thể
- Kể tên được các nhánh chính của AI và một số ứng dụng AI tại Việt Nam
- Thao tác được Google Colab: tạo notebook, chạy cell, đổi runtime
- Viết lại được biến, list, dict, vòng lặp, hàm trong Python
- Tự chỉnh sửa được một đoạn code chatbot đơn giản

TIẾT 1 — TRẢI NGHIỆM: KHÁM PHÁ AI QUA DEMO

Mở notebook "Demo_Buoi1.ipynb" (đã có sẵn toàn bộ code bên dưới). Với mỗi demo, bạn chỉ cần: copy code → paste vào một cell Colab → nhấn Shift+Enter để chạy. Sau đó suy nghĩ, thử cải tiến, và ghi lại thu hoạch.

Demo 1: Chatbot rule-based

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
rules = {
    "xin chào": "Chào bạn! Mình là trợ lý AI đơn giản.",
    "bạn là ai": "Mình là một chatbot dựa trên luật (rule-based).",
    "tạm biệt": "Hẹn gặp lại bạn!"
}

def chatbot(cau_hoi):
    cau_hoi = cau_hoi.lower().strip()
    return rules.get(cau_hoi, "Mình chưa được dạy để trả lời câu này.")

print(chatbot("Xin chào"))
print(chatbot("Bạn là ai"))
print(chatbot("Hôm nay trời đẹp không"))
```

💭 **Suy nghĩ: Vì sao chatbot trả lời được 2 câu đầu nhưng câu thứ 3 thì không?**

Đây là chatbot theo luật (rule-based): nó không hiểu nghĩa, chỉ so khớp nguyên chuỗi sau khi chuẩn hóa.

Trong hàm:

Python

```
cau_hoi = cau_hoi.lower().strip()
return rules.get(cau_hoi, "Mình chưa được dạy...")
```

- "Xin chào" → "xin chào" → **có** trong rules
- "Bạn là ai" → "bạn là ai" → **có** trong rules
- "Hôm nay trời đẹp không" → "hôm nay trời đẹp không" → **không có** key này

---

Nên câu 3 rơi vào giá trị mặc định.

Khóa "tạm biệt" cũng đã có sẵn, nhưng lần chạy đó không gọi nên không thấy.

Điểm hạn chế: chỉ cần khác 1 dấu cách, dấu hỏi, hoặc cách diễn đạt ("chào bạn", "bạn là ai vậy?") là không khớp.

🔧 Cải tiến: Hãy thêm ít nhất 3 cặp câu hỏi–trả lời mới vào dictionary rules (ví dụ: "bạn khỏe không", "mấy giờ rồi"…), rồi chạy thử lại. Dán code sau khi cải tiến của bạn vào khung bên dưới:

✏️ *Dán code chatbot đã cải tiến (thêm luật mới) của bạn vào đây*

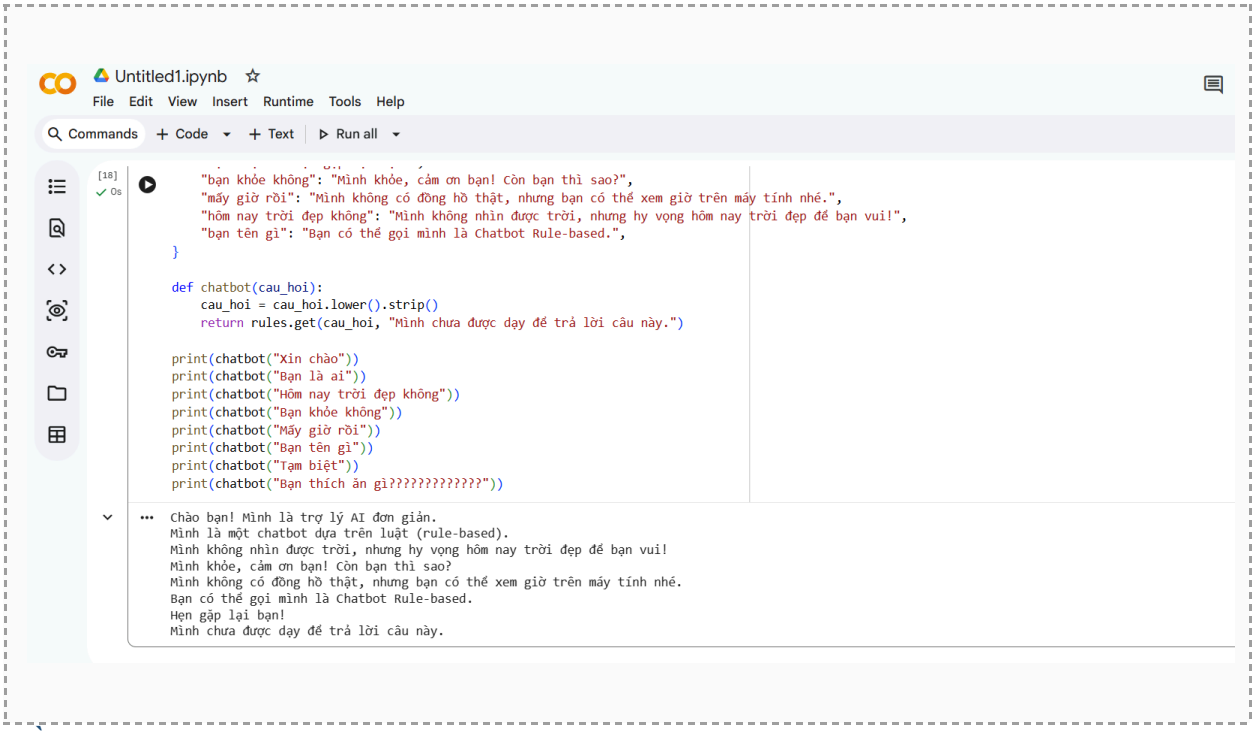
```
rules = {
    "xin chào": "Chào bạn! Mình là trợ lý AI đơn giản.",
    "bạn là ai": "Mình là một chatbot dựa trên luật (rule-based).",
    "tạm biệt": "Hẹn gặp lại bạn!",
    "bạn khỏe không": "Mình khỏe, cảm ơn bạn! Còn bạn thì sao?",
    "mấy giờ rồi": "Mình không có đồng hồ thật, nhưng bạn có thể xem giờ trên máy tính nhé.",
    "hôm nay trời đẹp không": "Mình không nhìn được trời, nhưng hy vọng hôm nay trời đẹp để bạn vui!",
    "bạn tên gì": "Bạn có thể gọi mình là Chatbot Rule-based.",
}

def chatbot(cau_hoi):
    cau_hoi = cau_hoi.lower().strip()
    return rules.get(cau_hoi, "Mình chưa được dạy để trả lời câu này.")

print(chatbot("Xin chào"))
print(chatbot("Bạn là ai"))
print(chatbot("Hôm nay trời đẹp không"))
print(chatbot("Bạn khỏe không"))
print(chatbot("Mấy giờ rồi"))
print(chatbot("Bạn tên gì"))
print(chatbot("Tạm biệt"))
print(chatbot("Bạn thích ăn gì"))
```

🌾 Thu hoạch: chụp màn hình kết quả trò chuyện với phiên bản chatbot đã cải tiến của bạn:

🖼️ *Dán ảnh screenshot cuộc trò chuyện với chatbot (bản đã cải tiến) tại đây*



Điều bạn rút ra được: Vì sao chatbot chỉ trả lời đúng khi câu hỏi khớp chính xác với luật đã có?

Chatbot này **không hiểu ngôn ngữ**, chỉ **tra từ điển**.

Câu hỏi được xử lý thế này:

1. Đưa về chữ thường và bỏ khoảng trắng thừa: cau_hoi.lower().strip()
2. Lấy đúng key đó trong rules bằng rules.get(...)
3. Có key thì trả lời theo luật; **không có thì ra câu mặc định**

Vì dict.get so sánh **chuỗi nguyên vẹn** (==), nên chỉ đúng khi câu hỏi sau chuẩn hóa **trùng 100%** với một key đã khai báo.

Ví dụ:

Câu người dùng	Sau .lower().strip()	Có trong rules?
Xin chào	xin chào	Có
xin chào!	xin chào!	Không (thêm dấu !)
Chào bạn	chào bạn	Không (khác wording)
Bạn là ai vậy	bạn là ai vậy	Không (thêm chữ vậy)

Đó là bản chất **rule-based / exact match**:

- Ưu: đơn giản, kiểm soát được câu trả lời, không cần model.
- Nhược: không nhận biến thể, không hiểu nghĩa, phải liệt kê từng câu một.

Muốn linh hoạt hơn thì phải đổi cách khớp: tìm **từ khóa con** ("chào" in cau_hoi), thêm nhiều biến thể cùng một ý, hoặc dùng model/NLP. Với code hiện tại thì “đúng luật thì đúng, lệch một ký tự là sai”.

Demo 2: Nhận diện chữ số viết tay

📋 Nội dung có sẵn — copy và chạy (mô hình đã được huấn luyện sẵn, bạn chỉ cần gọi để dự đoán):

```
# Mô hình đã được huấn luyện sẵn — chỉ gọi để dự đoán
prediction = model.predict(sample_image)
print("Mô hình dự đoán đây là chữ số:", prediction)
```

💭 **Suy nghĩ: Trước khi chạy, dự đoán của bạn là số mấy? Sau khi chạy, mô hình dự đoán số mấy?**

Dự đoán của tôi là số 6, sau khi chạy mô hình dự đoán số 6.

🔧 Cải tiến: Thử đổi sample_image sang một ảnh số viết tay khác (giảng viên cung cấp thêm 2–3 ảnh mẫu) và chạy lại để xem mô hình có luôn đoán đúng không:

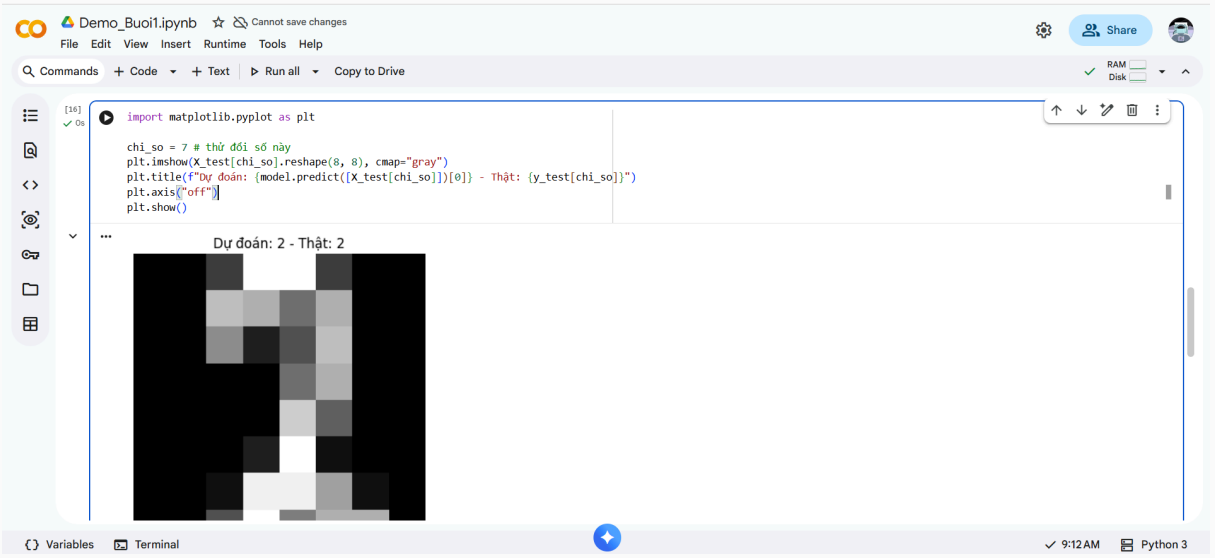
✏️ *Dán dòng code bạn dùng để thử ảnh số khác vào đây*

```
import matplotlib.pyplot as plt

chi_so = 7
plt.imshow(X_test[chi_so].reshape(8, 8), cmap="gray")
plt.title(f"Dự đoán: {model.predict([X_test[chi_so]])[0]} - Thật: {y_test[chi_so]}")
plt.axis("off")
plt.show()
```

🌾 Thu hoạch: chụp màn hình kết quả dự đoán trên ảnh mới:

🖼️ *Dán ảnh kết quả demo nhận diện chữ số (với ảnh mới) tại đây*



Demo 3: Hệ gợi ý phim bằng luật if-else

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
def goi_y_phim(the_loai):
    if the_loai == "hanh dong":
        return "Gợi ý: John Wick"
    elif the_loai == "tinh cam":
        return "Gợi ý: La La Land"
    else:
```

```
    return "Xin lỗi, mình chưa có gợi ý cho thể loại này"

print(goi_y_phim("hanh dong"))
print(goi_y_phim("hai huoc"))
```

🔧 Cải tiến: Hãy thêm ít nhất 2 nhánh elif mới cho các thể loại khác (hài hước, kinh dị, hoạt hình…):

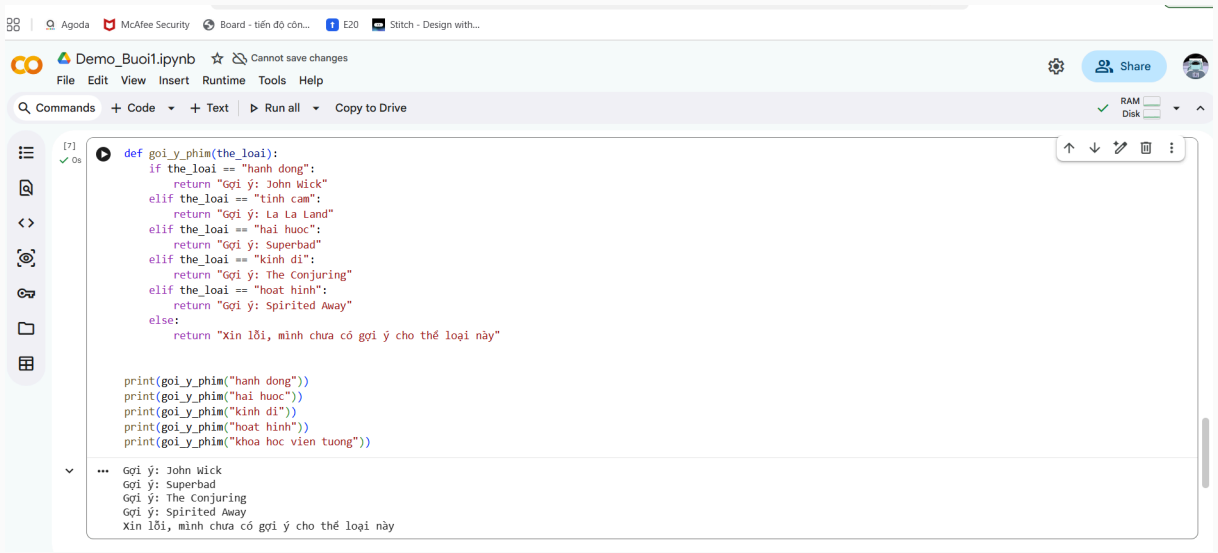
✏️ *Dán code hàm goi_y_phim đã cải tiến của bạn vào đây*

```
def goi_y_phim(the_loai):
  if the_loai == "hanh dong":
    return "Gợi ý: John Wick"
  elif the_loai == "tinh cam":
    return "Gợi ý: La La Land"
  elif the_loai == "hai huoc":
    return "Gợi ý: Superbad"
  elif the_loai == "kinh di":
    return "Gợi ý: The Conjuring"
  elif the_loai == "hoat hinh":
    return "Gợi ý: Spirited Away"
  else:
    return "Xin lỗi, mình chưa có gợi ý cho thể loại này"


print(goi_y_phim("hanh dong"))
print(goi_y_phim("hai huoc"))
print(goi_y_phim("kinh di"))
print(goi_y_phim("hoat hinh"))
print(goi_y_phim("khoa hoc vien tuong"))
```

🌾 Thu hoạch: chụp màn hình kết quả thử hệ gợi ý phim với các thể loại mới bạn vừa thêm:

🖼️ *Dán ảnh kết quả thử hệ gợi ý phim (bản đã cải tiến) tại đây*



Câu hỏi thảo luận: Theo bạn, trong 3 demo trên, demo nào 'thông minh' nhất? Vì sao?

Demo **nhận dạng chữ số (SVM)** thông minh nhất. Hai demo kia chỉ **khớp luật sẵn** (dict / if-elif): lệch một chút là sai. Demo chữ số **học từ dữ liệu** rồi đoán ảnh mới, đo được độ chính xác (~98.9%). Đó là machine learning.

TIẾT 2 — KHÁI QUÁT HÓA: GHI CHÚ LÝ THUYẾT

Vừa nghe giảng vừa điền vào các mục dưới đây bằng lời của chính bạn (không chép nguyên văn slide):

AI là gì? (định nghĩa theo cách hiểu của bạn):

AI là máy được lập trình hoặc huấn luyện để làm việc vốn cần trí tuệ người: trả lời câu hỏi, nhận ảnh, gợi ý, ra quyết định.

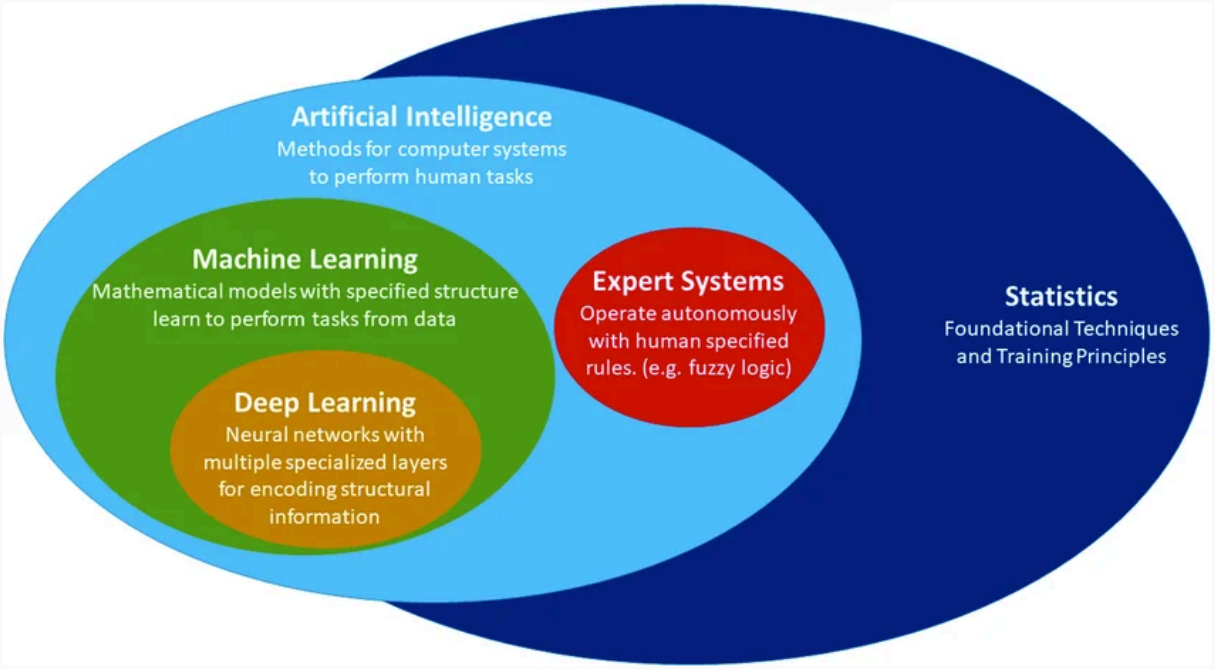
Sự khác biệt giữa AI, Machine Learning và Deep Learning:

AI: khái niệm rộng — máy mô phỏng khả năng thông minh.

ML: nhánh của AI — máy tự học quy luật từ dữ liệu, không cần liệt kê hết luật. Demo SVM chữ số thuộc nhóm này.

DL: nhánh của ML — dùng mạng nơ-ron nhiều lớp, mạnh với ảnh, tiếng nói, ngôn ngữ.
Tóm lại: DL ⊂ ML ⊂ AI.

Dán ảnh sơ đồ Venn AI/ML/DL (chụp từ slide) tại đây



3 mốc lịch sử AI mà bạn thấy ấn tượng nhất:

1950 — Turing Test: đặt câu hỏi "máy có nghĩ được không?".

2012 — AlexNet thắng ImageNet: deep learning nhận ảnh vượt hẳn cách cũ.

2022–nay — ChatGPT / LLM: AI hội thoại dùng được đại trà, gần với chatbot nhưng học từ dữ liệu lớn chứ không chỉ if/elif.

---

3 ứng dụng AI tại Việt Nam mà bạn biết hoặc vừa được giới thiệu:

Zalo: app nhắn tin của VNG.

FPT.AI: nền tảng AI của FPT.

VinBigData: thuộc Vingroup, làm AI dữ liệu / y tế.

TIẾT 3 — ÁP DỤNG CÓ HƯỚNG DẪN: COLAB & ÔN PYTHON

Danh sách thao tác cần hoàn thành (đánh dấu ✓ khi xong):

- ✓ Tạo notebook mới trên Google Drive → Colaboratory
- ✓ Phân biệt được cell code và cell markdown, dùng Shift+Enter để chạy cell
- ✓ Đổi runtime sang GPU (Runtime → Change runtime type)
- ✓ Lưu notebook về Google Drive của mình

📋 Nội dung có sẵn — copy và chạy lần lượt các đoạn code sau, quan sát kết quả in ra:

```
ten = "Colab"          # chuỗi (str)
so_luong = 15          # số nguyên (int)
diem_trung_binh = 8.5  # số thực (float)
dang_hoc = True        # kiểu boolean
```

```
danh_sach_diem = [8.5, 7.0, 9.2, 6.8]
print("Điểm cao nhất:", max(danh_sach_diem))

sinh_vien = {"ten": "An", "lop": "AI01", "diem": 8.5}
print(sinh_vien["ten"], "-", sinh_vien["lop"])
```

```
def phan_loai_diem(diem):
    if diem >= 8:
        return "Giỏi"
    elif diem >= 6.5:
        return "Khá"
    else:
        return "Trung bình"

for diem in danh_sach_diem:
    print(diem, "->", phan_loai_diem(diem))
```

📋 Nội dung có sẵn — hai hàm xử lý danh sách điểm, copy và chạy tiếp:

```
def tinh_trung_binh(danh_sach):
    return sum(danh_sach) / len(danh_sach)

def dem_gioi(ds_sinh_vien):
    dem = 0
    for sv in ds_sinh_vien:
        if sv["diem"] >= 8:
            dem += 1
    return dem

print("Điểm trung bình:", tinh_trung_binh(danh_sach_diem))
```

💭 **Suy nghĩ: Kết quả tinh_trung_binh in ra có đúng như bạn tự tính nhẩm không?**

Kết quả tinh_trung_binh in ra đúng như tôi tự tính nhẩm.

🔧 Cải tiến: Hãy thử cải tiến hàm tinh_trung_binh để làm tròn kết quả đến 2 chữ số thập phân (gợi ý: dùng hàm round()), và cải tiến dem_gioi để đếm thêm cả số sinh viên loại "Khá":

✏️ *Dán code 2 hàm đã cải tiến của bạn vào đây*

```
danh_sach_diem = [8.5, 7.0, 9.2, 6.8]

ds_sinh_vien = [
    {"ten": "An", "lop": "AI01", "diem": 8.5},
    {"ten": "Binh", "lop": "AI01", "diem": 7.0},
    {"ten": "Chi", "lop": "AI01", "diem": 9.2},
    {"ten": "Dung", "lop": "AI01", "diem": 6.8},
]

def tinh_trung_binh(danh_sach):
    return round(sum(danh_sach) / len(danh_sach), 2)

def dem_gioi_va_kha(ds_sinh_vien):
    dem_gioi = 0
    dem_kha = 0
    for sv in ds_sinh_vien:
        if sv["diem"] >= 8:
```

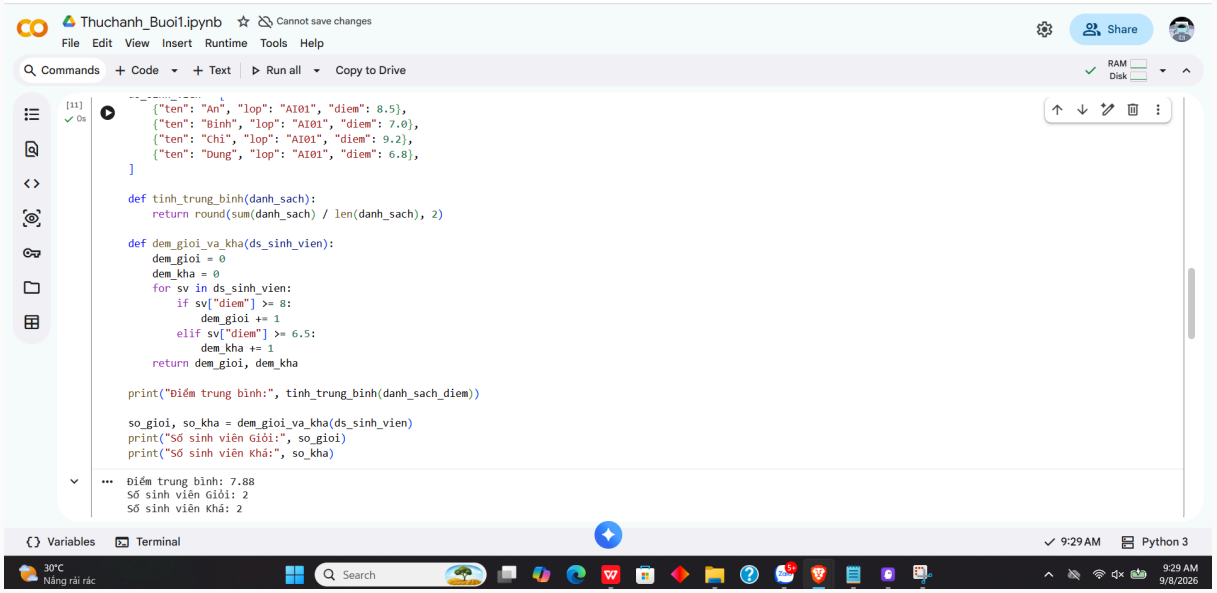
```
            dem_gioi += 1
        elif sv["diem"] >= 6.5:
            dem_kha += 1
    return dem_gioi, dem_kha

print("Điểm trung bình:", tinh_trung_binh(danh_sach_diem))

so_gioi, so_kha = dem_gioi_va_kha(ds_sinh_vien)
print("Số sinh viên Giỏi:", so_gioi)
print("Số sinh viên Khá:", so_kha)
```

🌾 Thu hoạch: chụp màn hình kết quả chạy các hàm sau khi cải tiến:

🖼️ *Dán ảnh kết quả chạy 2 hàm sau khi cải tiến*



TIẾT 4 — ÁP DỤNG MỞ RỘNG: BÀI TẬP NHÓM CHATBOT

Làm việc theo nhóm 2–3 người.

📋 Nội dung có sẵn — copy và chạy đoạn code chatbot_v2 sau (đã xử lý được câu hỏi theo từ khóa, không cần khớp chính xác toàn câu):

```
def chatbot_v2(cau_hoi):
    cau_hoi = cau_hoi.lower()
    if "gio" in cau_hoi:
        return "Mình chưa có đồng hồ, bạn xem giờ trên máy nhé!"
    elif "thoi tiet" in cau_hoi:
        return "Mình chưa kết nối dữ liệu thời tiết."
    else:
        return rules.get(cau_hoi, "Mình chưa được dạy để trả lời câu này.")

print(chatbot_v2("May gio roi"))
print(chatbot_v2("thoi tiet hom nay the nao"))
```

✓ Chạy thử chatbot_v2 với ít nhất 5 câu hỏi khác nhau trong nhóm

🔧 Cải tiến: Cả nhóm cùng thảo luận và thêm ít nhất 3 nhánh elif mới với từ khóa do nhóm tự chọn (ví dụ: "tên", "tuổi", "môn học"…):

✏️ *Dán code chatbot_v2 đã được nhóm bạn cải tiến (thêm từ khóa mới) vào đây*

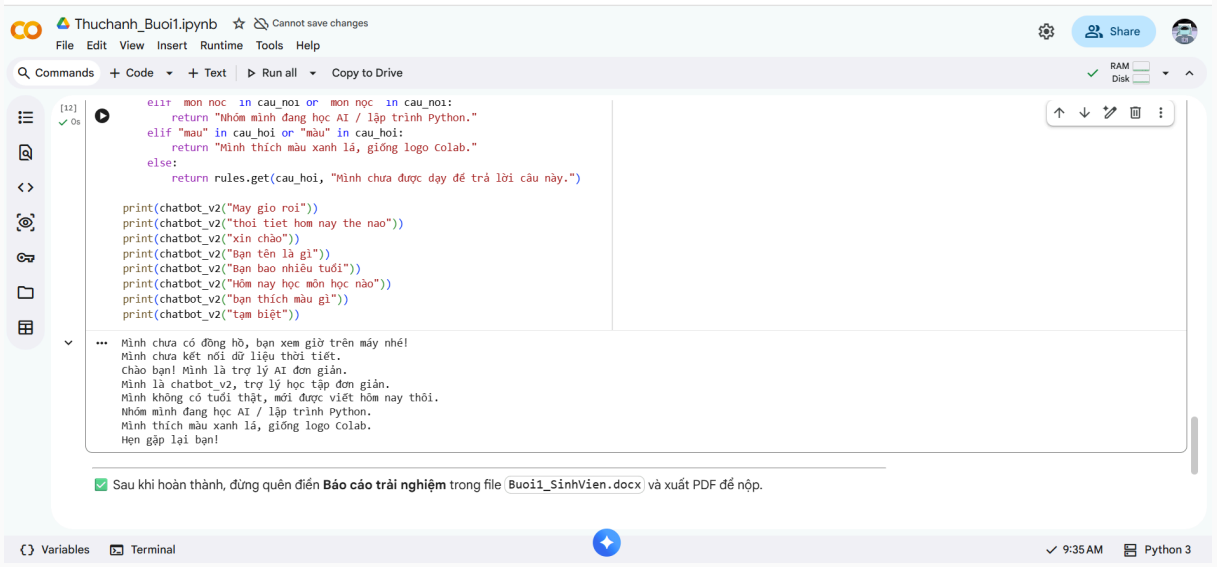
```
rules = {
  "xin chào": "Chào bạn! Mình là trợ lý AI đơn giản.",
  "bạn là ai": "Mình là một chatbot dựa trên luật (rule-based).",
  "tạm biệt": "Hẹn gặp lại bạn!"
}

def chatbot_v2(cau_hoi):
  cau_hoi = cau_hoi.lower()
  if "gio" in cau_hoi:
    return "Mình chưa có đồng hồ, bạn xem giờ trên máy nhé!"
  elif "thoi tiet" in cau_hoi:
    return "Mình chưa kết nối dữ liệu thời tiết."
  elif "ten" in cau_hoi or "tên" in cau_hoi:
    return "Mình là chatbot_v2, trợ lý học tập đơn giản."
  elif "tuoi" in cau_hoi or "tuổi" in cau_hoi:
    return "Mình không có tuổi thật, mới được viết hôm nay thôi."
  elif "mon hoc" in cau_hoi or "môn học" in cau_hoi:
    return "Nhóm mình đang học AI / lập trình Python."
  elif "mau" in cau_hoi or "màu" in cau_hoi:
    return "Mình thích màu xanh lá, giống logo Colab."
  else:
    return rules.get(cau_hoi, "Mình chưa được dạy để trả lời câu này.")

print(chatbot_v2("May gio roi"))
print(chatbot_v2("thoi tiet hom nay the nao"))
print(chatbot_v2("xin chào"))
print(chatbot_v2("Bạn tên là gì"))
print(chatbot_v2("Bạn bao nhiêu tuổi"))
print(chatbot_v2("Hôm nay học môn học nào"))
print(chatbot_v2("bạn thích màu gì"))
print(chatbot_v2("tạm biệt"))
```

🌾 Thu hoạch: chụp màn hình đoạn hội thoại thử nghiệm phiên bản chatbot đã cải tiến của nhóm:

🖼️ *Dán ảnh đoạn hội thoại thử nghiệm chatbot_v2 (bản đã cải tiến) của nhóm bạn*



Từ khóa mới thú vị nhất mà nhóm bạn đã thêm:

Từ khóa mới đã thêm: "ten" / "tên", "tuoi" / "tuổi", "mon hoc" / "môn học", "mau" / "màu".

Một câu hỏi mà chatbot của nhóm bạn vẫn trả lời sai/chưa tốt:

"Bạn đang học môn gì vậy?" — dễ trả lời sai. Nếu gõ không dấu thành "bay gio hoc mon gi" thì nhánh "gio" chạy trước, chatbot trả lời về đồng hồ thay vì môn học.

BÁO CÁO TRẢI NGHIỆM CUỐI BUỔI

Hoàn thành mục này trước khi xuất PDF và nộp bài:

1. TRẢI NGHIỆM — Tôi đã chạy/thử nghiệm đoạn code/bài toán nào? Kết quả quan sát được là gì?

Đã chạy 4 phần: chatbot rules (khớp nguyên câu), gợi ý phim goi_y_phim bằng if/elif, SVM nhận dạng chữ số, xử lý điểm + chatbot_v2 theo từ khóa.

Phim: "hanh dong" → John Wick, "hai huoc" → Superbad, "kinh di" → The Conjuring, "hoat hinh" → Spirited Away; thể loại lạ rơi vào else. Chatbot v1 chỉ đúng khi trùng key. SVM ví dụ chi_so = 7 đoán 2 (đúng). Trung bình điểm 7.875 → làm tròn 7.88. chatbot_v2 bắt được "May gio roi", "thoi tiet hom nay the nao".

2. QUAN SÁT – PHẢN ÁNH — Kết quả có giống dự đoán ban đầu của tôi không? Điều gì khiến tôi bất ngờ hoặc chưa hiểu rõ?

Giống dự đoán: phim/chatbot lệch chuỗi là sai. Bất ngờ: SVM đoán đúng ảnh 8×8 mờ, không cần luật từng ảnh. Chưa rõ lúc đầu: phải print mới hiện kết quả. chatbot_v2 dễ sai vì if trên thắng, ví dụ "bay gio hoc mon gi" bị trả lời về giờ.

3. KHÁI QUÁT HÓA — AI là gì? AI/ML/DL khác nhau như thế nào? (diễn đạt bằng lời của chính mình)

AI: máy làm việc giống cần trí tuệ người.

ML: học từ dữ liệu rồi đoán cái mới.

DL: ML dùng mạng nơ-ron nhiều lớp.

AI là máy mô phỏng trí tuệ; ML là AI biết học từ dữ liệu; DL là ML dùng mạng nơ-ron nhiều lớp — DL nằm trong ML, ML nằm trong AI.

4. ÁP DỤNG – RÚT KINH NGHIỆM — Nếu gặp bài toán tương tự, tôi sẽ thử làm gì khác? Tôi còn thắc mắc điều gì?

Bài giống phim/chatbot: thêm nhiều nhánh + từ khóa, thử câu lệch chính tả. Bài giống chữ số: tách train/test, thử nhiều mẫu.

Còn thắc mắc: làm chatbot hiểu nghĩa; SVM sai ảnh nào và vì sao.

Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — (5)

✅ Sau khi hoàn tất, vào File → Download → PDF Document để xuất tài liệu này (kèm toàn bộ ảnh và ghi chú của bạn) và nộp theo hướng dẫn của giảng viên.